



TravalBee

Product Requirements Document — V1 Launch

EXECUTIVE BRIEF · CONFIDENTIAL · APRIL 2026

Executive Vision

TravalBee is the world's first **deterministic AI travel concierge** — a bespoke digital service that behaves less like a chatbot and more like a seasoned luxury travel editor who has already been everywhere, knows everyone, and has already called ahead.

Where competitors surface inspiration, TravalBee delivers **execution**. A single interaction produces a complete, geographically coherent, day-by-day itinerary: activities, curated dining, hotel recommendations, transit estimates, and hidden gems — all verified against live Google Places data, rendered in a split-screen editorial interface that would not look out of place in *Vogue* or *Condé Nast Traveller*.

The **Zero-Latency Principle** is the architectural cornerstone: every detail — coordinates, opening hours, transit times, photo assets, and affiliate hotel links — is generated and enriched in a **single upfront pipeline call**. There is no second request. No waiting. No stale data. The result is a product that feels instantaneous, works offline, and scales without accumulating API debt.

The word “AI” does not appear anywhere in the user interface. TravalBee is positioned around the luxury **outcome**, not the technology. The engine is invisible. The experience is everything.

“The engine is invisible. The experience is everything.”

Market Positioning & The Problem

The Problem with Generic AI Travel Tools

The current landscape of AI-assisted travel planning suffers from three fatal flaws:

- **Hallucination without consequence** — General-purpose chatbots fabricate restaurants, invent opening hours, and recommend venues that closed years ago. There is no grounding mechanism.
- **Geographical incoherence** — Without spatial constraints, AI itineraries routinely schedule a morning in one district and an afternoon 90 minutes away by car, then back again.
- **Inspiration without execution** — Most tools stop at suggestions. None produce a structured, actionable plan with real coordinates, verified hours, and bookable hotel links.

The TravalBee Differentiator

-
- **The Neighbourhood Lock** — A hardcoded spatial constraint injected into every AI prompt. Walking-mode itineraries are locked to a single city; car-mode limits consecutive stops to $\leq 40\text{km}$ / ≤ 30 min. Geographically incoherent output is architecturally impossible.
 - **Google Places Enrichment** — Every activity and restaurant cross-referenced against the Google Places API: photos, live ratings, review counts, opening hours, and price levels.
 - **Structured JSON Contract** — The AI response is a strictly typed JSON schema validated by Zod. The pipeline either produces a valid itinerary or self-heals via a repair call before surfacing anything to the user.
 - **Model Selection is Non-Negotiable** — Claude Sonnet 4.6 is the sole supported model. Claude Haiku was evaluated and rejected: it produces spatially incoherent output on Neighbourhood Lock + negative constraint tasks. Documented in the engineering changelog.

Product Strategy — The Freemium Wedge

TravalBee employs a frictionless freemium model engineered for maximum top-of-funnel conversion. The paywall is encountered only *after* the user has experienced the product's full value.

Feature	Free Tier	Paid Tier — \$4.99
Itineraries / month	5	Unlimited
Max trip duration	3 days	5 days
Credit card required	No	One-time
Credits expire	—	Never
Google Maps & enrichment	✓	✓
PDF export & sharing	✓	✓
Promo code support	—	✓

The Conversion Funnel

Step	Route	Function
1	/	Editorial hero — “BEGIN YOUR JOURNEY” CTA
2	/sample	Public no-auth showcase itinerary — organic acquisition hook
3	/curate	10-field staged intake form
4	/itinerary	Live split-screen results; 402 paywall modal on credit exhaustion
5	/pricing	Single \$4.99 credit purchase via Stripe Checkout
6	/dashboard	Credits widget, world map, trip statistics

The 402 Gate

When a user exhausts credits, POST `/api/itinerary` returns **HTTP 402**. The client detects this status code and renders a paywall modal — never an error state. The experience remains premium throughout.

Stripe Webhook Credit Upsert

```
checkout.session.completed → userProfile.upsert
```

```
First-time buyer: create { availableCredits: 2 } // preserves unused free credit
```

```
Returning user: update { increment: 1 }
```

Core User Experience

The Intake Form — 10 Fields, Zero Friction

A staged inline expansion form that unlocks sequentially. Each stage reveals only after the previous is confirmed — reducing cognitive load and guiding the user toward a complete, high-quality brief:

#	Field	Detail
01	Destination	Google Places Autocomplete
02	Travel Dates	Departure + return, local timezone-safe
03	Travel Party	Solo / Couple / Family / Group
04	Pace	Relaxed 3–4/day · Moderate 4–5/day · Packed 6–7/day
05	Budget Tier	Premium \$\$ · Luxury \$\$\$ · Ultra-Luxury \$\$\$\$
06	Dietary Preferences	7 options (None, Vegetarian, Vegan, Halal, Kosher, GF, DF)
07	Interests	10 options, no cap
08	Accommodation	Need a hotel / Already booked
09	Hotel Name	Places Autocomplete (lodging) — verified address as spatial anchor
10	Transport Mode	Walking & Transit / Private Car · Driver + Walking Tolerance sub-option

The Results Interface

Split-screen layout: **55% editorial timeline + 45% interactive map.**

- Timeline: chronological day cards with activity photos, ratings, opening hours, transit connectors, and meal recommendations interwoven in a single pass
- Map: day-centric SVG markers (champagne / slate / midnight / emerald / rose), semantic activity-type icons, polyline routing, dynamic day filter
- Tabbed day navigation with AnimatePresence cross-fades
- InteractiveStays hotel slider: 6 AI-generated hotels across 3 tiers, filterable by rating, Booking.com affiliate links (AID 4013143)

Offline, Shareable, Printable

- PWA-installable via @serwist/next — service worker, standalone display
- Full offline capability via localStorage vault (seek_wander_archive)
- /shared/[id] — public, no auth, acquisition banner, mobile sticky CTA

-
- PDF export: window.print() + Tailwind print modifiers — zero dependency

Technical Architecture & SRE

Technology Foundation

Layer	Technology
Framework	Next.js 14 App Router, TypeScript
Styling	Tailwind CSS v3 (custom design tokens, rounded-none everywhere)
Animation	Framer Motion 12
Database	Supabase PostgreSQL + Prisma 7 (pooled + direct dual URLs)
Auth	Clerk v6 — modal-only, no dedicated auth pages
AI Model	Anthropic claude-sonnet-4-6, max_tokens 8192
Maps	@react-google-maps/api (split key: browser + server)
Payments	Stripe — one-time checkout, HMAC webhook
Rate Limiting	Upstash Redis — slidingWindow(5, "1h") per userId
PWA	@serwist/next + serwist
Observability	@sentry/nextjs — wizard-installed, full stack instrumentation
Deployment	Vercel — serverless, edge-ready, daily Cron cleanup

The Single-Request Data Pipeline

1. Zod validation → reject malformed input before any AI/DB call
2. Upstash rate limit → 429 on exhaustion (5 req / hr / userId)
3. Credit check → 402 if availableCredits < 1
4. claude-sonnet-4-6 → structured JSON itinerary (max_tokens: 8192)
5. JSON self-healing → sanitizeJson() + Anthropic repair call if needed
6. Places enrichment → PlaceCache-first (30-day TTL, 100-entry cap)
7. Haversine transit → pure local math, \$0 API cost
8. CostLog.create() → awaited — Vercel freeze protection
9. Response.json() → complete itinerary delivered

Observability — Enterprise-Grade Sentry Integration

- @sentry/nextjs installed via official wizard; withSentryConfig wraps next.config.mjs
- Full stack coverage: server config, edge config, instrumentation (client + server), global-error boundary
- layout.tsx uses export function generateMetadata() — enables per-request trace correlation via Sentry.getTraceData()

- AI pipeline outer catch uses `Sentry.withScope({ destination })` — all exceptions tagged with destination context for rapid triage
- `AbortError` intentionally excluded — client navigation cancellations suppressed to maintain signal-to-noise ratio

Security Architecture — 33-Point Audit Complete

Layer	Mechanism
Input Validation	Zod <code>safeParse()</code> on all POST bodies — rejects before AI/DB
Prompt Injection	Blocklist regex on all user-supplied fields injected into AI prompt
IDOR Prevention	<code>findUnique</code> + <code>userId</code> check; neutral <code>notFound()</code> for missing AND unauthorized
Rate Limiting	<code>slidingWindow(5, "1h")</code> per <code>userId</code> via Upstash Redis
API Key Split	Browser key (HTTP referrer restricted) vs server key (Places API only)
Stripe Webhook	HMAC signature via <code>stripe.webhooks.constructEvent()</code>
HTTP Headers	X-Frame-Options: DENY, nosniff, Referrer-Policy, Permissions-Policy
Cron Security	Authorization: Bearer — 401 for all other callers
Admin Route	<code>notFound()</code> not redirect — route existence not leaked to non-admin

Precision Unit Economics

Every itinerary generation writes a `CostLog` record to Supabase with exact, micro-cent precision. Token counts are extracted directly from the Anthropic SDK response — no estimation, no multipliers, no rounding.

The CostLog Schema

Field	Type	Description
<code>inputTokens</code>	Int	Exact input tokens from Anthropic API
<code>outputTokens</code>	Int	Exact output tokens from Anthropic API
<code>thinkingTokens</code>	Int	Exact adaptive thinking tokens (when enabled)
<code>aiCost</code>	Decimal(10,6)	$(\text{input} \times \$3 + \text{output} \times \$15) / 1,000,000$
<code>googleCost</code>	Decimal(10,6)	$\text{cacheMisses} \times \$0.032 + \text{detailCalls} \times \0.017
<code>totalCost</code>	Decimal(10,6)	<code>aiCost</code> + <code>googleCost</code>
<code>cacheHits</code>	Int	PlaceCache hits — \$0 Google API cost
<code>cacheMisses</code>	Int	Fresh Google Places calls

Token Cost Rates

	Rate	Unit
Claude Input Tokens	\$3.00	per 1,000,000 tokens
Claude Output Tokens	\$15.00	per 1,000,000 tokens
Google Places Text Search	\$0.032	per API call
Google Places Details	\$0.017	per API call
PlaceCache hit	\$0.000	30-day TTL, 100-entry cap

Margin Protection Architecture

- **GenerationMeta** (full cost breakdown) logged to server console only — **never transmitted to the client**. Margin data is not exposed in browser developer tools.
- `prisma.costLog.create()` is **awaited** before `Response.json()` — Vercel freezes the serverless function the instant the HTTP response returns, killing any background promise. Awaiting guarantees zero financial data loss.

-
- PlaceCache-first enrichment dramatically reduces Google API spend: a warm cache on a popular destination approaches **\$0 in Google costs** per generation.

Admin Observability — /admin/metrics

Access controlled by ADMIN_USER_ID env var. notFound() (not redirect) for all non-admin access — route existence is never leaked.

- KPI cards: Total platform spend · Total generations · Avg cost per trip · Cache hit rate
- Cost split: Claude vs Google with per-trip averages
- Generation log: last 200 rows, colour-coded (green <\$0.15, amber >\$0.50)
- Revenue KPIs: total purchases, revenue, average order value
- User metrics: total users, active users, totalGenerations per user

TravalBee is production-ready. The architecture is battle-tested, the economics are transparent to the micro-cent, and the user experience is designed for a customer who expects nothing less than exceptional. This document reflects the V1 launch state.